小项目实施指南：面试弹药库

0. 通用要求（2026-10-23 收口）

1. 每个项目都要有 README（写清设计取舍、实验数据、复现命令）：独立仓库的放自己仓库里，S5/S6 这种并进 P1 的放 `experimental/` 下；M1、M5 只写笔记，不开仓库；M2 已删除，留档不执行。代码是写给你自己看的，真正的弹药是 README 和实验记录；
2. 每个项目都必须攒齐"弹药三件套"：一个设计取舍、一个亲自踩过的坑、一张数据或对照表；笔记也一样，至少要有一条能说出口的结论；
3. 硬性时间盒：开工前在 README 顶部写下截止日期和止损日；到止损日就砍功能，不能砍质量，更不许对自己说"再打磨一周"；
4. 弹药纪律：项目做完之后立刻写一页《复盘卡》（如果重来会怎么改、被追问时最薄弱的三个点），面试之前你只需要看这张卡；
5. 名额纪律：独立占用工期的项目可以全做，不做"二选一"；但必须遵守互补原则——每个项目只做大项目没覆盖的空白域，不重复；
6. 微弹药 M1–M5：M3 已升级为 1 周的独立项目，M4 要开独立仓库，M1/M5 只写笔记，M2 删除；它们仍放在第 9 节，编号不变；
7. 回写纪律：每做完一个，当天往 P1 或 P2 的 README 塞一句话或一张图。成果不许只堆在本地目录里；
8. 产出纪律：每个小项目至少产出一条主项目能用到的数字或结论，否则降级成笔记，不算项目；
9. 止损纪律：到止损日立刻收工，不追求完整度。目标是可讲的证据，不是可发布的产品。

1. 互补全景图（v4 新增，先读这一节再看每个项目）

这一节列出 P1/P2 覆盖了什么、没覆盖什么，每个小项目补的是哪块空白。这是 v4 重写的核心决策依据。

P1 TrainCache 覆盖的能力域

能力域	P1 实现方式	小项目补什么
存储数据面	CAS 内容摘要 + 驱逐 + P2P + **重扫描恢复**	S4 补 **WAL + merge + WAL replay 恢复**（和 P1 的重扫描正交）

K8s 调度	**数据亲和**（前缀就绪度门控 + 打分 + SchedulingGate）	S6 补 **拓扑亲和**（NFD/topology-updater/NUMA/GPU zone）
Agent 观测	**用户态**（inotify + /metrics）	S3 补 **内核级**（eBPF tracepoint + block I/O 直方图）
容器数据面	hostPath 只读挂载（preheat 模型）	M3 补 **CSI 标准 API**（NodeStageVolume + NodePublishVolume/bind mount）
分布式共识	用 etcd（不自研 Raft）	S1 补 **自研 Raft**（选举/复制/快照/崩溃恢复）
传输层	**gRPC stream** 做主传输；Transport 接口抽象出来但 RDMA 实验臂未实现	S5 补 **RDMA verbs 手写实现**（通信原语选型 + Go↔C++ 跨语言桥接 + Soft-RoCE 验证）

P2 KVRouter 覆盖的能力域

能力域	P2 实现方式	小项目补什么
KV cache 视角	**网关视角**（消费 vLLM 推的 KV 事件）	S7 补 **引擎内核视角**（精读 block_manager.cpp + 写简化版）
推理调度	消费已完成调度的结果（continuous batching 在 vLLM 内核里）	M5 补 **精读 scheduler**（continuous batching / preemption 调度逻辑）
观测粒度	**粗粒度**（vLLM /metrics 的 TTFT/TBT/吞吐）	M4 补 **token 级**（eBPF uprobes 精确到每个 token 的 prefill/decode 时间）
引擎源码理解	消费 AIBrix 路由层源码	S7 + M5 补 **引擎内核精读**（block_manager + scheduler + attention）

小项目编号与决策总表（2026-10-23 收口）

编号保持旧版不变（外面有引用），删掉的项目只打标记、不删段。

项目	补什么	定位	这次的决定	产物放哪
~~**S1 mini-Raft KV**~~	分布式共识：选举、日志复制、快照、崩溃恢复	P1 论证	改造保留，3 周止损	独立仓库，作为 P1 附录（编号等 P1 目录定稿后再补，不占 §9.2）
S2 io_uring 预取器	C++ 系统编程：io_uring、O_DIRECT、对齐内存	P1 实验臂	不动	并入 P1，不占独立名额
S3 eBPF 观测工具	内核级观测：块 I/O 直方图、major fault、预读归因	P1 缺口	保留	独立仓库，作为 P1 附录

S4 bitcask mini	WAL 持久化、merge、WAL replay 恢复	P1 缺口	改造保留：只留 WAL / merge / 崩溃恢复	独立仓库，作为 P1 附录
S5 RDMA 传输臂	RDMA 数据面、通信原语选型、Go↔C++ 桥接	P1 路线验证	保留但强止损，4 周	并进 P1 仓库 experimental/
S6 K8s 调度插件	拓扑亲和：NFD + topology-updater + 官方 NRT 插件	P1 边界	保留：只做配置与链路，砍阶段 B	并进 P1 仓库 experimental/
S7 mini KV Cache Manager	引擎内 KV block 管理：固定大小池、LRU、前缀查找	P2 缺口	保留	独立仓库，作为 P2 附录
M1 mini-container	容器隔离认知	认知补充	删减：只写一篇笔记	不开仓库
M2 mini-CNI	容器网络认知	认知补充	删除，不设复活条件	留档，不执行
M3 mini-CSI	CSI 标准 API、kubelet 注册链路	P1 缺口	改造保留，1 周	独立仓库，作为 P1 附录
M4 token 级 profiling	token 级观测	P2 视角桥	保留	独立仓库，作为 P2 附录
M5 vLLM 引擎源码精读	引擎内核精读	P2 底座	保留	笔记，P2 引用

一句话原则：补漏洞的项目（S3、S4、M3、S7）单独留仓库；当实验臂的项目（S5、S6）并进 P1 的 experimental/；只做论证的项目（S1）最后落成 P1 里的一句话；M4 单开仓库作为 P2 附录；M1、M5 只留笔记，M2 不做。

执行顺序（2026-10-23 收口）

固定顺序只有一条：P2 线 **M5 精读笔记 → S7 简化版 block manager → M4 token 级 profiling**，不能颠倒。M5 决定 S7 的设计依据，也决定 M4 的探针挂在哪。

P1 线按接入时机走：S4 等 P1 的存储层能跑重扫描恢复之后做；S3 在 P1 核心 1 完工之后做；M3 等 P1 能把 hostPath 换成 CSI 时做；S6 阶段 C 跟 P1 步骤 7 合流；S5 挂在 P1 步骤 11 之后。S1 独立推进，3 周止损。

2. S1 mini-Raft KV（必做，3 周硬顶）

互补关系：P1 用 etcd（不自研 Raft）、P2 是单机网关——S1 补的是分布式共识的工程经验，这是大项目都不覆盖的能力域。

介绍：用 Go 实现一个 3 节点的 Raft，对外暴露 Get/Put 接口。做这个项目的目的，是把一致性从"背出来的知识点"变成"能讲清楚的东西"。你要保持清醒：这是 MIT 6.824 里人手一个的项目，实现本身的差异化几乎为零，简历上永远不要把它排在第一位；它的弹药价值全在工程细节、实验设计，以及和 etcd 的对拍里。

定位（2026-10-23 收口）：这是一道论证题，不是功能题。你要回答的是"P1 为什么用 etcd、不自研 Raft"，不是把 Raft 做成一个能上线的库。

范围：必须做选举、日志复制、快照、崩溃恢复的最小可用集；做故障注入（kill -9、网络分区、时钟跳变）；和 etcd 跑同一串操作做行为对拍。不做：membership 变更、Pre-Vote（这两个列为"你必须能在白板上讲清楚"的内容）。

止损（2026-10-23 收口）：3 周。对拍报告写完就停，不补边角特性；如果 P1 的元数据后端能临时换成它，再跑一轮 benchmark 就收工。

关键设计（保留 v2）

- 状态机三个角色，用一个单 tick 驱动所有定时器（time.Timer 的超时时间在 [T, 2T) 内随机化，T 大约取 500ms）；
- 有三个字段必须持久化：`currentTerm`、`votedFor`、`log[]`——这是一个经典的死亡陷阱：`votedFor` 不落盘，节点就可能在同一个任期里投出两票，进而脑裂。WAL 的记录格式是 `len | crc32 | entry`，采用追加写；
- RPC（gRPC）包括 `RequestVote` 和 `AppendEntries`：收到每个回复都先比 term，发现自己的 term 更小就立刻让位——这是第二大陷阱；
- 提交规则：leader 只能提交当前任期的日志（对应 §5.4.2）；prevLogIndex/term 不匹配时回退 nextIndex，如果你实现了冲突项的加速回退，这是加分项；
- 线性一致读（stretch 目标）：用 ReadIndex 或走日志的方式实现；你要能讲清楚"为什么 ReadIndex 对时钟的依赖比 lease 小"；
- 快照：在 `lastIncludedIndex` 之前截断日志，配合 `InstallSnapshot` 传输；恢复时先装快照，再重放后续日志。

时间够再做的三组实验（原 v3 增强，2026-10-23 降级）

> 下面三组从"必做"降为"时间够再做"。止损日一到就砍，不影响收工。

1. **工业对读（时间够再做）**：toy 版本跑通之后，精读 etcd 的 raft 库——重点看 `node.go` 里的 Ready 循环、unstable log、Storage 接口，以及 ReadIndex 的工业实现。最后产出一篇《我的 toy raft 与 etcd raft 的 5 个差异》（候选差异：批量化的 Ready 通道、消息流水线、leader 转移、检查点时

机、无环日志结构）。"写过一个 lab"和"写过 lab 还读过工业实现"的人，在面试里完全是两个物种；

2. **实验一换血（时间够再做）**：删掉"30% 丢包单点"这个老实验（它太极端了，只能得到"它瘫了"这一个结论），改为丢包率扫描 0.1% / 1% / 5% / 10%，画出 commit P50/P99 随丢包率变化的曲线，并解释曲线的拐点（重传与选举超时是如何耦合的）；
3. **实验二为新增内容（时间够再做）**：无主停机窗口的分布——`kill -9` leader 至少 100 次，记录从 kill 发生到新 leader 完成第一个成功写入的时间分布（中位数/P99）。这是一个比提交延迟更受分布式面试官看重的可用性数字。分区实验要区分三种：多数派隔离、少数派隔离、不对称分区（leader 单向失联），分别给出行为表。

实施步骤

1. 单机 KV 加 WAL；2. 搭 gRPC 骨架和选举（打印每次 term 变化和投票流向）；3. 复制与提交（把 raftd 集成进测试 harness，注入延迟、分区和重启）；4. 快照和 `kill -9` 恢复；5. 故障注入（kill -9、三类网络分区、时钟跳变）+ 和 etcd 跑同一串操作做对拍；6. 时间够再跑丢包扫描和无主窗口分布；7. 可选：把 P1 元数据后端临时换成它跑一轮 benchmark。

验收标准（2026-10-23　　收口）：故障注入下，已经提交的数据零丢失；三类分区的行为表完整；和 etcd 的同序列对拍报告写完（同一串操作，逐项比较返回值、日志顺序、恢复后的数据）。丢包扫描曲线、无主窗口分布、etcd 五差异笔记降为选做。

回写 P1 README：写一句话："元数据层对比自研 Raft 与 etcd，结论见附录。"如果 P1 元数据后端替换的 benchmark 跑了，再补一张对比表；没跑就注明只做了行为对拍。

你能讲的点：`votedFor` 为什么必须持久化；为什么只能提交当前任期的日志；ReadIndex 与 Lease 的对比；你的 toy 实现与 etcd 的具体差异；P1 为什么最终选 etcd；Pre-Vote 如何防止分区节点拿着旧 term 扰乱选举；joint consensus 两阶段成员变更（不要求实现，但要能白板讲）。

---

3. S2 C++/io_uring 预取器（并入 P1，不占独立名额）

互补关系：P1/P2 主力 Go，无 C++ 系统编程——S2 补的是 C++ + io_uring + O_DIRECT + aligned memory 这套系统级硬货。同时是 vLLM C++ 源码精读的前置训练（S7/M5 需要读 vLLM 的 C++ 代码，S2 先让你熟悉 C++ 系统编程的坑）。

定位：P1 §9.2 A 轨（小文件高 IOPS 臂）的实验分支。可以为它单独建仓库，但不占小项目名额，也不另排独立工期，随 P1 步骤 8 前后推进即可。

范围：做本地写路径和提交模型；不做：完整的 S3 客户端（HTTP Range 已经够用）。

关键设计（保留 v2）

- io_uring（liburing）：队列深度在 64–256 之间可配；用 `IORING_REGISTER_FILES` 加 registered buffer 消掉每个 op 的固定开销；

- O_DIRECT 三重对齐的坑：缓冲区地址、长度、偏移都必须对齐逻辑块（用 `posix_memalign` 对齐到 4K）；
- overlayfs 的坑：在容器里的 overlayfs 上用 O_DIRECT，可能直接报 EINVAL——目标路径必须落在宿主机真实的 ext4/xfs 上（用 hostPath，或者干脆在宿主机上直接跑）；
- 提交与收割：`get_sqe → prep → submit_and_wait(N)`，CQE 批量收割之后再 resubmit，错误码逐个处理（短读即 EOF）；
- 用 fio 校准：先测出裸盘直写的上限，然后贴着这个上限比，"我比 fio 差 8%"这种说法比"我很快"专业十倍。

v3 补强

- 步骤 0 先 `uname -r` 锁定内核版本（registered files/buffers 的行为会随版本变化），并且把 liburing 官方 examples 逐个通关——io_uring、O_DIRECT、overlayfs 三个坑叠加在一起，直接上手太陡了，而 C++ 是你的次语言，这个坡度必须削平；
- 收益预期要预先诚实化：P1 主臂的 4MiB 块经 HTTP 抓取时受网络限速，本地写路径省下的 syscall 对吞吐的贡献预期接近于零；io_uring 真正的战场是 A 轨的小文件高 IOPS。报告里正确的写法是"syscall 数量大幅下降、大块吞吐不变（并解释为什么受网络限速）、小文件臂 IOPS 提升 X"，不许只挑好看的数字报。

实施步骤

1. Go 线程池加 buffered 基线 → 2. C++ buffered 版本（隔离语言因素）→ 3. O_DIRECT 加 io_uring → 4. registered 优化版 → 5. 跑参数矩阵（QD × 块大小 4K/64K/1M × 并发，记录吞吐、strace 统计的 syscall 数、CPU 占用）。

弹药：O_DIRECT 对齐与 overlayfs 的坑；io_uring 到底减少了什么（syscall 加线程调度）；为什么大块场景下它赚不到收益——能把"不适用的边界"讲清楚，比吹收益值钱得多。

---

4. S3 eBPF 观测工具（可选，后置，MVP 从单个探针扩到四件事）

互补关系：P1 用 **用户态 inotify** 观测、P2 用 vLLM /metrics 观测——S3 补的是 **内核级** 观测，能看到用户态看不到的东西（比如 readahead 造成的 major fault 不会记在消费者进程名下，按进程维度归因就会算错）。

定位：后移到 P1 核心 1 完工之后，有余力了再做。MVP 不再只做一个探针：块 I/O 延迟直方图、major fault 计数、预读命中/浪费归因、和 P1 inotify 指标对齐，四件事围绕同一轮 benchmark 做，不铺开做全家桶。

范围：只做只读观测；要求内核版本不低于 5.10；在宿主机上跑（容器权限很麻烦，先在宿主机上做）。

2026-10-23 收口：下面前两件和最后两件都算 MVP，缺一件就不算完成；第三个探针（read 字节

量）仍是可选。

v3 范围

- **第一件（必做）**：block I/O 延迟直方图——用 tracepoint `block/block_rq_issue` 加 `block/block_rq_complete`，按 request 指针配对，在内核侧按 log2 bucket 聚合，用户态周期性拉取 per-CPU map；
- **第二件（必做）**：major fault 计数（`exceptions/page_fault_user` 里的 major/minor 位）；
- **第三件（可选）**：`sys_enter_read` 的字节量；
- **第四件（必做）**：预读命中/浪费归因——统计预读进来的页有多少被真正用上、多少被浪费，最后给出"预读是帮忙还是帮倒忙"的结论；
- **第五件（必做）**：和 P1 的 inotify 指标对齐——把文件事件、dataloader 打点、内核块 I/O 和 major fault 对到同一条时间线上，做一张映射表；
- 有一条观测逻辑必须写进报告：P1 的 preheat 模型下，缓存命中 page cache 时根本不会产生 block I/O——所以 block I/O 直方图天然就能区分命中与未命中；但这里有一个 **readahead 陷阱**：内核预读造成的 major fault 不会记在消费者进程名下，按进程维度归因就会算错。这个陷阱本身就是上等弹药；
- CO-RE：用 `vmlinux.h` 加 bpf2go；优先用 tracepoint、少用 kprobe（ctx 字段在跨版本时更稳）；准备一个降级方案：采样 `/proc/diskstats`，并诚实标注两个口径的差异。

实施步骤

1. 跑 bpf2go 的 hello-world 验证环境 → 2. 做 block I/O 配对延迟直方图加 Go 聚合 → 3. major fault 探针（必做），read 字节量探针（可选）→ 4. 做预读命中/浪费归因，把 inotify 指标、dataloader 打点和内核观测对到同一条时间线 → 5. 把 exporter 接入 P1 实验，同一轮 benchmark 下比较用户态观测和内核观测的停顿时间差异 → 6. 往 P1 README 回写归因图。

验收数字（2026-10-23 新增）：同一轮 benchmark 下，停顿时间在用户态观测（inotify / dataloader 打点）和内核观测（块 I/O + major fault）之间的差异；预读到底帮了忙还是帮了倒忙的结论。

回写 P1 README：一张「预取命中率归因图」，标清瓶颈落在页缓存还是块设备。

验收标准：在 P1 实验期间能稳定出数；两个口径交叉验证一致；readahead 归因陷阱有专门一节来讲。

弹药：per-CPU map 与 ringbuf 的取舍；为什么 page cache 命中时没有 block I/O；readahead 如何污染进程归因；CO-RE 解决了什么问题。

5. S4 bitcask mini 存储引擎（v4：聚焦 WAL/merge/crash recovery，与 P1 正交）

互补关系（v4 核心修正）：P1 用 CAS + **重扫描** 做 crash recovery（重启后扫 `blocks/` 目录，按路径重建索引，~分钟级），不做 WAL、不做 merge、不做增量恢复。S4 补的是 **WAL 持久化 + merge/compaction + WAL replay 增量恢复**（重启后 replay WAL 重建索引，~秒级），以及 fsync 三档对比、单写多读模型的内存账。

> **面试时的对比故事**："P1 的 CAS 存储层靠重扫描做 crash recovery（实现简单，但重启慢），S4 我用 WAL 做增量 recovery（重启快，但 merge 逻辑复杂）——两种 crash recovery 路径我都实践过，知道各自的适用边界。"

介绍：用 Go 实现一个 bitcask：WAL 加内存索引，加 hint file，加 merge。范围：只做单机；不做：分布式（那部分由 S1 覆盖）、完整的 LSM（与它对读即可）。

形态（2026-10-23 收口）：独立仓库，作为 P1 附录。只留 WAL、merge、崩溃恢复三件事，CAS、refcount、P2P 块共享不碰（这些是 P1 的）。必须交出的数字：重启恢复时间（目标秒级，对比 P1 的分钟级）、fsync 三档的吞吐/延迟/丢数据风险对照表、merge 前后的读放大倍数。10M key 的内存账继续做。

关键设计（v4 修改：砍掉与 P1 重合的部分）

> **v4 不再实现**：CAS contentHash 存储层（路径寻址 + 内容哈希摘要）、refcount 引用计数、P2P 块共享——这些是 P1 的东西，S4 不用。S4 用 WAL + CRC 做完整性校验，不做 CAS。

- **WAL 记录格式**（保留 v2）：`crc32 | tstamp(8B) | keysz(4) | valsz(4) | key | value`，tombstone 用 valsz 的特殊值标记（删除操作同样是一次写）；tstamp 统一用 int64 毫秒级（主动在注释里写出"4 字节秒级时间戳有 2038 问题"——小细节，但能留下大印象）；
- **内存索引**：`map[key] → {fileID, offset, size}`；10M 个 key 的内存账，你要在自己机器上实测出数字（key 平均长度 × key 数量 × Go map 的开销）——这条账决定了 bitcask 的适用边界；
- **活跃文件写满（256MB）之后转为 immutable**；
- **fsync 三档做成配置**：逐条刷盘 / group commit（N 毫秒）/ 只靠操作系统——把三档拿来 benchmark 对比，这是存储引擎最核心的权衡之一；
- **merge**：把 immutable 文件中的存活记录写入新数据文件和 hint file，然后原子切换：先 rename 数据文件和 hint 文件，最后写 manifest 指向新的文件集合；你要把每一个崩溃窗口想清楚，写进注释；
- **读路径**：索引命中之后用 `pread` 读取，再做 crc 校验（校验失败意味着盘上数据腐坏，这时报错或者触发重新 merge）；
- **WAL replay 增量恢复**（v4 强调）：重启时从 manifest 找到最后一次合并位置，replay 之后的 WAL 记录重建内存索引——这是和 P1 重扫描恢复的核心差异。

v3 的四处补强（保留 v3）

1. **时间戳改成 8 字节**：主动在注释里写出这个修正；
2. **benchmark 要打到 10M key 这一档**（32G 内存的机器跑得动）：1M 档算不出 bitcask 的命门——全内存索引的容量账，你必须在 10M 档下实测启动时间、常驻内存、读写吞吐，画出"索引内存/数据量"的比值曲线；
3. **明确实现单写多读模型，并且把它讲透**：写锁天然串行化（这是 bitcask 和 LSM 共同的核心约束），读操作无锁；压测多写者的排队情况和多读的扩展性；
4. **加一次 LSM 对读（不要求实现）**：读 LevelDB/RocksDB 的 compaction 文档，最终目标是能在白板上回答"数据量超过内存之后，bitcask 为什么会输给 LSM（对比 SSTable、稀疏索引、分层 compaction 的读写放大）"。

实施步骤

1. append-only log 加读路径 → 2. 多文件轮转加 tombstone → 3. **启动恢复对比**：先暴力扫全目录，再用 WAL replay，对比两种方式的启动时间（秒级 vs 分钟级）→ 4. merge 加 manifest 原子切换 → 5. 用 fuzz 随机注入 `kill -9`，跑完崩溃矩阵 → 6. benchmark：1M/10M 两档、merge 期间的写延迟抖动（再丑的数字也要敢贴）、写放大和**读放大**（merge 前后，读一份有效数据平均要从磁盘读多少份）、fsync 三档对比 → 7. **对比实验**：把 P1 的重扫描恢复数据和 S4 的 WAL replay 数据放在同一张表里，量化两种 crash recovery 的差异 → 8. 往 P1 README 回写结论。

步骤 8：回写 P1 README

写一句话："P1 采用重扫描做 crash recovery（重启 ~分钟级），WAL 方案见附录 S4，未上主路径的原因是 X。" X 要写具体——工程复杂度、merge 的崩溃窗口、维护成本，三选一或都写，不能写"没时间"。

验收标准（2026-10-23 收口）：崩溃矩阵全绿；10M 档的容量账有实测；merge 抖动数据完整；**启动恢复时间对比表**有实测；fsync 三档的吞吐/延迟/丢数据风险对照表完整；merge 前后的读放大倍数有数字；LSM 对读笔记一页；P1 README 回写完成。

弹药三件套：
- 取舍：WAL replay vs 重扫描（实现复杂度 vs 恢复速度）；merge 原子切换的崩溃窗口；fsync 三档的性能 vs 持久化保证
- 坑：overlayfs 上 O_DIRECT 的坑（和 S2 联动）；merge 期间的写延迟抖动
- 数据：10M key 内存账；fsync 三档吞吐/延迟/丢数据风险对照表；**启动恢复时间对比表**（P1 重扫描 vs S4 WAL replay）；merge 前后读放大倍数

6. S5 RDMA 传输臂：Soft-RoCE + ibverbs + C++/Go 桥接（并进 P1 experimental/，4 周止损）

> 2026-10-20 新增：原目录 S4 与 S6 之间空着一个序号。将 P1 中标记为「保留 v3」、正文里只写了

> 「永远不进主路径」就没了的 RDMA 实验臂内容独立成章。P1 PDF 保持不变。

互补关系：P1 的主路径用 gRPC stream 做数据传输（§8.2 Transport 接口默认实现），简单可靠、生态成熟、调试方便。RDMA 作为实验臂挂在 P2P 块共享后面，v4 正文里只写了「永远不进主路径」就没了——Transport 接口抽象出来了，但 RDMA 那个实现没人写过。S5 补的是：真正手写 ibverbs 代码跑通 RDMA 数据面、通信原语选型决策（Send/Recv vs RDMA Write）、以及 Go↔C++ 跨语言桥接。这三件事 P1 里只提了一句「别用 cgo 链进 Go 热进程，用独立 C++ 组件加 C ABI」，但为什么这样、桥接怎么设计、Go 侧指针怎么处理——全是空白。

> **面试时的对比故事**：「P1 用 gRPC stream 做主力传输（可靠、好 debug、生态全），S5 我用 RDMA verbs 写了 P2P 实验臂（内核旁路、零拷贝、CPU 零开销，但工程复杂度完全不是一个量级）——两套传输栈我都写过，知道什么时候该用哪个，也知道 RoCE 在跨子网时的无损以太网坑。」

介绍：在两台 Linux 虚拟机上用 Soft-RoCE（内核模块，不需要买 Mellanox 网卡）跑通真正的 RDMA verbs 代码，实现一个对标 P1 Transport 接口的块数据传输原型。做这个项目的目的，是把 P1 里「挂个 Transport 接口」从设计图变成真正跑在网卡上的数据通路。

范围：只做 Soft-RoCE 功能验证；不做 GPUDirect RDMA（没有 GPU）；不进 P1 主路径（RDMA 传输臂永远是 P2P 的可选后端）。

不做：eRDMA 云真机校准（这是真值数据，按小时租阿里云 eRDMA 卡一下就行，几十块钱，不属于本项目工期）、GPUDirect RDMA、RDMA 进 P1 主路径替换 gRPC。

2026-10-23 收口：不再单独建仓库，代码并进 P1 仓库的 experimental/。这是一个强止损的实验臂：4 周。必须交出四样东西——verbs 手写（含 Send/Recv vs RDMA Write 的选型说明）、内存注册与内存池、Go↔C++ 桥接用独立 C++ 组件 + C ABI（不把 cgo 链进热进程）、Soft-RoCE 功能验证。到期如果只做到"跑通"，降级成一篇《为什么我没在主路径上 RDMA》，不进主路径。

关键设计

- **① C++ 写 ibverbs 封装层，暴露 C ABI，Go 侧不 cgo 链进热进程**：这是 P1 明确指定的架构约束，也是本项目第一个核心决策。原因有三：（1）Go 的 cgo 调用有固定开销——goroutine 切换到 C 线程约 2–5μs，对延迟敏感的 RDMA completion 路径来说不可接受；（2）ibverbs 的 completion queue polling 放在 C 进程里跑更干净，轮询循环直接在 C++ 守护进程里阻塞，不占 Go runtime 调度；（3）Go 进程的 GC 可能移动内存，传给 ibverbs 的 buffer 如果由 Go 分配，GC 一搬就炸。C++ 侧用 `malloc` 分配 + `ibv_reg_mr` 注册，生命周期完全可控；
- **② 通信原语选型：P2P 场景下 RDMA Write 优先于 Send/Recv**：P2P 块共享是「拉取式」——节点 A 需要一个块，去找 peer B 要。用 RDMA Write：peer B 预注册好几个块槽位的内存区域，发布 rkey；peer A 拿到 rkey 后，用一条 RDMA Write 直接把数据写到 B 的槽位里，B 的 Go 进程甚至不用配合（写完后通过 completion queue 通知 B 的守护进程）。如果用 Send/Recv：peer B 需要提前 post receive buffer，双方配合完成传输，跟「对方无感知」的设计目标冲突——而且 Send/Recv 是双边操作，需要处理更复杂的同步逻辑。gRPC 默认用 Send/Recv 语义（RPC 天然请求-响应），RDMA 实验臂用 Write 语义，这就是两套传输栈的核心差异；

- **③ 内存注册是 RDMA 的核心门槛：大块预注册 + 内存池复用**：`ibv_reg_mr`（注册内存区域）开销大——几百微秒到毫秒级，因为内核要锁定物理页、建立虚拟到物理地址的映射、Pin 住页面防止 swap。所以不能按需注册小块数据（4KB 一块注册一次反而比 TCP 还慢）。正确做法：启动时大块预注册（几 MB 到几十 MB），Go 侧维护一个内存池，注册一次重复使用。内存池的设计：C++ 侧暴露一个 C ABI（分配/释放/查询剩余空间），Go 侧用 `sync.Pool` 管理正在使用的 buffer——但这些 buffer 必须从 C 分配的内存里拿，不能用 Go 的 `make([]byte)`；
- **④ CQ 轮询 vs 事件通知：Soft-RoCE 上轮询更稳定**：Completion Queue 有两种取事件的方式：轮询（`ibv_poll_cq` 不停扫）和事件通知（epoll + `ibv_get_cq_event`）。Soft-RoCE 是内核态的软件模拟，对中断的支持不完整——事件通知经常丢事件，导致请求挂起超时。真正的 Mellanox/NVIDIA 网卡上，轮询也比事件延迟更低（少一次中断），但 CPU 占用从 0% 跳到 100%。你要做一组对比数据：Soft-RoCE 上两种方式的稳定性差异、CPU 占用差多少、延迟差多少。这是面试时证明「我真跑过 RDMA」的硬证据；
- **⑤ RoCE 依赖无损以太网：PFC + ECN + MTU=4096**：InfiniBand 端到端是自己的协议栈，保证所有包都到，丢包不重发。RoCE 把 IB 传输层搬到以太网上，但以太网天生会丢包——所以 RoCE 有三个硬性前提：PFC（Priority Flow Control，缓冲区快满时暂停发送）、ECN（Explicit Congestion Notification，队列有点堵时提前标记通知发送方减速）、MTU 设成 4096（Jumbo Frame，减少分片开销）。Soft-RoCE 同机/同网段能直接跑（不依赖 PFC/ECN），但跨子网就会丢包且不重发，RDMA 直接认为包都到了——然后挂死。面试时要能画清楚 PFC 和 ECN 怎么配合让以太网变「无损」。

实施步骤

1. **Soft-RoCE 环境搭建**（2 小时）：两台 Ubuntu VM（内核 ≥ 5.x），加载 softroce 模块（`modprobe softroce`），配好同网段 IP，用 `rping -s 12345 -c` 跑通 RDMA Ping。这一步是后面所有事的闸门——rping 不通就别往下走；
2. **最小 ibverbs demo**（半天）：纯 C++ 写一个 client/server，跑通 RDMA Write——client 注册内存、post send（`opcode=IBV_WR_RDMA_WRITE_WITH_IMM`），server 注册内存、等待 completion。不需要 Go，先把 ibverbs API 本身搞定。重点理解 `ibv_create_qp` / `ibv_qp_rig_init` / `ibv_post_send` / `ibv_poll_cq` 这几个核心函数；
3. **C ABI 封装 + Go 桥接**（1–2 天）：把 ibverbs 代码封成一个共享库，暴露 3–4 个简单函数：`init()`（打开设备、创建 PD/CQ/QP、注册 MR）、`write(peer_rkey, peer_addr, local_buf, len)`（发起 RDMA Write）、`poll(timeout_ms)`（阻塞等待完成，返回 0/非 0 表示成功/超时）、`shutdown()`。Go 侧用 `// #include` 加 `import "C"` 调用这些函数，但注意：不要把 ibverbs 的任何内部状态（PD/CQ/QP/MR）暴露给 Go 侧——Go 只管调高层接口，所有复杂状态都在 C++ 里；
4. **对标 P1 Transport 接口**（半天）：把 C ABI 包装成 Go 的 Transport 接口实现。P1 §8.2 里 Transport 是 Go 的 `interface`，默认实现是 gRPC stream。你要做的是：再写一个 `rdmaTransport` struct，实现同样的 `SendChunk` / `ReceiveChunk` 方法，内部调 C ABI。P2P 块共享在初始化时注入 Transport 实现——gRPC 或 RDMA，运行时无感知；
5. **P2P 块传输原型**（1 天）：两台节点各起一个 RDMA 守护进程（C++）+ 一个 Go peer 服务，节点

A 预注册 8 个 1MB 块槽位的 MR，发布 rkey 给节点 B。节点 B 收到「需要块 X」的请求，用 RDMA Write 把本地块写到 A 的槽位里。完成后 A 的守护进程从 CQ 轮询到事件，通知 Go 侧「块 X 就绪」。这就是 P1 P2P 块共享场景下 RDMA 实验臂的完整数据通路；

6. **性能对比实验**（半天）：Soft-RoCE vs TCP（Go net 标准库）vs gRPC stream，三个实现并排对比：块大小扫 4KB / 256KB / 4MB 三档，每档跑 1000 次取 P50/P99 延迟和吞吐。**诚实预期**：Soft-RoCE 常常比 TCP 还慢（因为是软件模拟），但你要把数据贴出来——这恰好是面试时「我知道 RDMA 在什么场景下才真正有价值」的弹药。另外测两组：CQ 轮询 vs 事件通知的 CPU 占用差异、`ibv_reg_mr` 注册耗时（证明为什么不能按需注册）；
7. **（可选，不在工期内）租阿里云 eRDMA 校准真值**：Soft-RoCE 是模拟器，性能数字不能进报告。真值数据按小时租阿里云 eRDMA 云主机，几十块钱就能拿到。校准一次就够了——对比 Soft-RoCE 和真 eRDMA 的延迟差距，这个差距本身就是弹药。但这一步不占工期，放在最后按预算推进。

验收标准：Soft-RoCE 下 RDMA Write 功能跑通（Step 5 的 P2P 块传输原型）；4KB / 256KB / 4MB 三档延迟吞吐数据完整；Go↔C++ 桥接代码有注释说明为什么不用 cgo 链进 Go 热进程；RoCE 无损以太网的 PFC / ECN 机制能讲清楚；`ibv_reg_mr` 注册耗时有实测数据（证明为什么必须预注册大块内存）。

回写 P1 README：Transport 接口下的第二条曲线，或一条明确的否定结论，二选一。只写"试过 RDMA"不算交付。

弹药三件套：
- 取舍：Send/Recv vs RDMA Write（双边 vs 单边，对端是否需要配合，P2P 拉取式场景天然适合 Write）；CQ 轮询 vs 事件通知（延迟最低 vs CPU 零占用，但 Soft-RoCE 上事件通知经常丢，轮询是被迫选择）；cgo 链进 Go 热进程 vs 独立 C++ 组件（工程复杂度 vs 性能与稳定性，P1 明确指定后者）；
- 坑：Go 分配的内存传给 `ibv_reg_mr` 会被 GC 移动，导致注册无效（解决方案：用 C.alloc 分配，或 Go 1.21+ 的 `runtime.Pinning` 固定内存）；Soft-RoCE 跨子网丢包不重发（不支持 PFC/ECN 时 RDMA 直接挂死）；`ibv_reg_mr` 按需注册 4KB 小块的开销比 TCP 还大（必须大块预注册 + 内存池）；CQ 事件通知在 Soft-RoCE 上丢事件导致请求挂起超时（只能轮询）；
- 数据：Soft-RoCE vs TCP vs gRPC 三档延迟吞吐对比表；`ibv_reg_mr` 注册耗时 vs 块大小曲线（证明预注册大块的必要性）；CQ 轮询 vs 事件通知的 CPU 占用对比；（可选）Soft-RoCE vs eRDMA 云真机的延迟差距。

面试时能讲的点：cgo 调用开销（goroutine → C 线程切换约 2–5μs）；`ibv_reg_mr` 的内核态开销和物理页 Pin 机制；为什么 RDMA Write 对 P2P 拉取式场景天然适配（单边操作、对端无感知）；RoCE 无损以太网的三个前提（PFC / ECN / MTU=4096）及其配合关系；Soft-RoCE 模拟 vs 真 eRDMA 的性能差距（几十倍）；Transport 接口抽象的工程价值（gRPC 默认 + RDMA 实验臂，运行时无感知切换）。

---

7. S6 K8s 调度插件实战：拓扑感知打分（并进 P1 experimental/，只做配置与链路）

互补关系（v4 核心修正）：P1 步骤 7 只做 **数据亲和**（前缀就绪度门控 + 打分 + SchedulingGate），没有 NFD/topology-updater/拓扑约束。S6 补的是 **拓扑亲和**——从硬件拓扑（NUMA/GPU zone）角度做 Filter/Score，和 P1 的数据亲和是正交的两个调度维度。

> **面试时的对比故事**："P1 做了数据亲和调度（前缀就绪度门控 + 打分），S6 我补了拓扑亲和调度——用 NFD 抓 NUMA 拓扑，用官方 NRT 插件做 Filter/Score，然后在 P1 的数据亲和插件上加了一列拓扑维度，做了双目标冲突实验。两条正交的调度维度我都有实践。"

介绍：基于 kubernetes-sigs/scheduler-plugins，跑通官方的拓扑感知调度，在"数据亲和 × 拓扑亲和"的冲突点上做增量实验。绝对不要声称"我发明了拓扑调度器"，官方 NRT 插件就是你的对照组。

2026-10-23 收口：只做配置与链路，不写自研插件。阶段 A 和阶段 C 的脚本、报告都并进 P1 仓库的 experimental/，不再单独建 S6 仓库。

范围（v4 修改）：
- ✓ **阶段 A**（保留，独立工期 ~1 周）：部署 NFD + topology-updater + 官方 NRT 插件全链路
- ✗ **阶段 B**（砍掉）：自研最小 PreFilter + Score 插件——和 P1 步骤 7 的数据亲和插件完全重复（P1 步骤 7 已经写了更完整的版本：数据亲和 + SchedulingGate + 门控 + 降级），再写一遍是重复劳动
- ✓ **阶段 C**（保留，与 P1 步骤 7 合流，不额外占工期）：双目标冲突实验

事实前提（保留 v3 不变）

- 官方已经提供 `NodeResourceTopologyMatch`（Filter）加 `NodeResourceTopologyResourceAllocationScore`（Score），它们消费 `NodeResourceTopology` CRD（KEP-119），由 NFD（Node Feature Discovery）加 topology-updater 负责上报；
- Topology Manager 从 k8s 1.27 开始 GA；
- DRA 在今天已经是既成事实，不要再把它当成"未来趋势"来讲：Dynamic Resource Allocation 从 Kubernetes v1.35 开始 GA（stable，特性门控锁定且不可关闭，v1.30 首次引入）；
  v1.36（2026-05）进一步把 ResourceClaim 接入了新的 Workload/PodGroup（内核 gang scheduling 加 topology-aware scheduling）体系。但 2026 年的现网现实是：NRT 插件仍然是主流，各厂商的 DRA driver 生态还处在迁移期——所以 S6 正确的叙事是"亲手实现旧设备模型，同时讲清新世界如何取代它"，两边的功夫都要硬。

DRA 白板要点（不写代码，与阶段 A 同期写半页笔记）

下面四条你必须能脱稿讲清楚：

1. 旧 device plugin 模型的痛点：设备被抽象成整数资源（比如 `nvidia.com/gpu: 1`），你无法按显存、型号或互联拓扑发出结构化请求，它不支持设备共享，也不支持按属性表达式过滤，拓扑信息只能走 Topology Manager + NFD + NRT 这条外挂链路；
2. DRA 的四个核心对象（`resource.k8s.io/v1`）：驱动用 ResourceSlice 上报设备能力与拓扑，集群

管理员定义 DeviceClass（可以类比 StorageClass），工作负载用 ResourceClaim / ResourceClaimTemplate 申领设备（体验类似 PVC/PV 的申领），调度器据此做结构化匹配，kubelet 与驱动之间完成分配握手；它支持 CEL 属性过滤和设备共享；
3. 它与 S6/NRT 的关系：DRA 让设备拓扑成为调度器可以直接消费的结构化属性，长期来看会收编 NFD+NRT 这条外挂链路的一部分；但迁移期两套机制并存，NRT 插件在现网中仍被广泛部署——你能讲清"哪部分会被取代、哪部分（NUMA/CPU 亲和）仍会独立存在、迁移卡在哪里（驱动成熟度与存量集群版本）"，才是满分答案；
4. 前沿联动：v1.36 的 Workload/PodGroup API 在内核里做 gang scheduling 加 topology-aware scheduling，并且接入了 DRA——这与 P1 的训练 Job 调度、S6 的拓扑亲和是同一个演进方向，面试时可以当作"我在跟踪内核调度演进"的谈资，但明确不跟进实现。

硬件诚实清单（开工第一件事）

先跑一次 `numactl -H` 确认：游戏本的消费级 CPU 几乎肯定是单 NUMA 节点，也没有多 GPU；所以所有拓扑都用 fake/annotation 驱动来开发——这完全正当，官方插件自己的 e2e 测试也在 mock NRT 对象；真机多 GPU 拓扑只在 P2 租云主机拿真值的那个窗口顺带读一次（`nvidia-smi topo -m`），不值得为它单独开预算。

阶段 A：现勘并跑通官方链路（约 1 周，v4 保留）

1. 搭好 kind 集群（版本对齐 scheduler-plugins 的兼容矩阵），按仓库 Developer Guide 在本地构建镜像（kind load）或者直接用发布的 chart；
2. 部署 NFD 加 topology-updater，生成真实的 `NodeResourceTopology` CR；在单 NUMA 机器上手写一份 fake NRT yaml（构造双 NUMA、fake GPU 资源池），作为第二个数据源；
3. 配置 v1 版本的 `KubeSchedulerConfiguration`：包括 profiles（schedulerName）、plugins（在 filter/score 上启用 NRT 插件）、pluginConfig（scoringStrategy：MostAllocated、BalancedAllocation、LeastAllocated 各跑一遍，观察决策差异）；
4. 部署若干请求 CPU/GPU 的测试 Pod，验证绑核与拓扑决策是否符合预期，抓取调度日志；
5. 精读：官方 Score 插件源码（分数如何由 NRT 的 zones 和资源分配算出、如何归一化）、NRT CRD 的字段语义；
6. **验收标准**（你能口述全链路）：device plugin 的 TopologyInfo → kubelet Topology Manager（与 CPU Manager 协同）→ NFD topology-updater → NRT CRD → 调度插件 Filter/Score → bind；
7. **风险保护**：如果一周之内官方全家桶部署不成功（NFD 组件多、镜像拉取坑也多），S6 立即降级为"手写 NRT yaml + 只部署插件本体"；再不行就降级为"纯跟读源码 + 写决策日志推演笔记"，不能让环境黑洞吃掉第二周。

阶段 B：自研最小插件（v4 砍掉，理由见上）

> **v4 为什么砍掉阶段 B**：P1 步骤 7 要写的调度插件比 S6 阶段 B 的"最小件"还完整——P1 有数据亲和 + SchedulingGate + 门控 + 降级，S6 阶段 B 只有拓扑亲和的最小件。做了 P1 步骤 7 之后，再做 S6 阶段 B 等于再写一遍同一个 framework 的同一种插件，只是打分公式换成另一种。面试时只能讲"我写过

> 两个调度插件"，讲不出差异。

阶段 C：双目标冲突实验（1–2 周，与 P1 步骤 7 合流，不额外占工期，v4 保留）

1. 让两个 Score 维度并存：**数据亲和分**（P1 的 Job 子集 readyRatio，由 P1 的 informer 缓存提供）× **拓扑亲和分**（S6 的 fake NUMA/GPU zone 匹配度，由 S6 阶段 A 的 NRT CRD 提供）；
2. 构造冲突场景：Job 的热分片缓存在 NUMA-A 节点上，但空闲 GPU 在 NUMA-A'（拓扑上靠近远端内存）；或者另一个节点 NUMA 局部性很好，但数据是冷的；
3. 权重 w 从 0 扫到 1，画出权重-调度决策曲线，以及对应的 makespan/拓扑惩罚代理指标；
4. 把终点问题写成报告：框架级的多插件分数如何归一化（各插件 Score 的值域）、冲突时为什么不存在银弹、生产环境中应该让谁把权重暴露给用户；
5. 合流动作：**S6 的拓扑亲和 Score 直接作为 P1 步骤 7 调度插件的第二个 Score 维度**——P1 仓库的主代码里保留完整的双维度插件，experimental/ 里保留阶段 A 的部署脚本、阶段 C 的实验报告和权重扫描曲线；
6. **验收标准**：产出一张冲突权重-决策曲线图；DRA 四条白板要点能脱稿讲（见上，包含 v1.35 已 GA 的现状判断，禁止再说"DRA 是未来的实验特性"）。

回写 P1 README：调度阶段 = 数据亲和 + 拓扑亲和，两格都填满。哪格空着就写清楚为什么。

弹药三件套

- 取舍（**v4 修改**）：NRT annotation fake 与 NFD 全家桶之间怎么选、双目标如何归一化、**为什么阶段 B 被 P1 步骤 7 覆盖了**（体现你能识别重复劳动、主动优化）
- 坑：KubeSchedulerConfiguration 的 apiVersion、多 profile、schedulerName、RBAC 权限、kind 镜像加载
- 数据：权重-决策曲线加 scoringStrategy 对照

诚实清单：拓扑全部是 fake 的；官方 NRT 插件是基线，不是要打倒的对手；没有做绑核验证，也没有真机性能数据；DRA 没有动手实现，但它的事实状态（v1.35 GA）和四个要点你必须跟到能讲、能回答迁移期共存问题的程度。

8. S7 mini KV Cache Manager（v4 新增，P2 互补）

互补关系：P2 把 vLLM 当黑盒用（只消费 KV 事件，不自研 KV block 管理）。S7 补的是 **引擎内核视角**——精读 vLLM 的 `block_manager.cpp` 之后，用 C++ 写一个简化版 block manager，管理固定大小的 KV cache blocks、LRU 驱逐、prefix lookup、capacity 动态调整。

> **面试时的对比故事**："P2 是网关视角看 KV cache（消费引擎推的事件），S7 是引擎内核视角看 KV cache（自己管理 block pool、驱逐、prefix 匹配）——两种视角我都有实践，知道引擎内部和网关之间怎么协作。"

时间盒：2–3 周（和 P2 步骤 4 并行）

形态（2026-10-23 收口）：独立仓库，作为 P2 附录。顺序固定在 M5 之后、M4 之前：M5 精读笔记 → S7 简化版 block manager → M4 token 级 profiling，不能颠倒。

前置：S2 的 C++ 系统编程经验（io_uring / O_DIRECT 之后你对 C++ 内存管理更熟了）、M5 的 vLLM 源码精读笔记（先读再写）

范围：做 block pool、LRU 驱逐、prefix lookup；不做：真正的 vLLM 内核、CUDA 代码、continuous batching 调度。

vLLM 内核精读（前置，M5 联动）

精读 vLLM 源码的三个核心文件（用 vLLM v1.x 锁定版本）：

1. `vllm/v1/core/kv_cache_manager.py`（或 `vllm/vllm/v1/worker/gpu_model_runner.py` 里的 block manager）：KV block 的分配、释放、淘汰、prefix cache 索引
2. `vllm/attention/backends/paged.py`：paged attention 的内存布局（每个 block 多大、token 怎么映射到 block）
3. `vllm/vllm/v1/core/scheduler.py`：continuous batching 调度时 KV cache 怎么被引用和释放

产出一篇《vLLM KV Cache 内核对读笔记》——重点记录三个问题：
- block 是多大？token 怎么映射到 block index？
- LRU 驱逐时怎么判断"这个 block 还能不能被某个 in-flight 请求引用"？
- prefix cache 的索引结构是什么？怎么判断"新请求的前缀命中了缓存里的哪些 block"？

C++ 简化版实现

精读完 vLLM 内核后，用 C++（C++17，不用 CUDA，跑在 CPU 上）写简化版 block manager：

核心数据结构：
- `Block`：固定大小（默认 16 blocks/页，可配置），每个 block 存一段 token 的 KV 数据（简化为固定大小的 buffer，不需要真 FP16/BF16 数据）
- `BlockPool`：`vector<Block>`，固定容量，初始化时全部标记为 free
- `PrefixIndex`：`map<BlockHash, vector<BlockID>>`——block 哈希（前 256 token/block 的递归哈希）到持有该 block 的 ID 列表
- `LRU`：`list<BlockID>` + `unordered_map<BlockID, list::iterator>`

核心接口（伪代码）：

```
class KVCacheManager {
public:
    // 从 pool 中分配 n 个连续 block（如果有 free list 就复用）
    vector<BlockID> AllocBlocks(int n);

    // 标记这些 block 被某个 request 引用（in-flight，不能驱逐）
    void RefBlocks(const vector<BlockID>& blocks, RequestID req);

    // request 结束后释放引用，如果没人引用了就放回 free list
    void UnrefBlocks(const vector<BlockID>& blocks, RequestID req);
```

```
    // LRU 驱逐：从 tail 开始淘汰，跳过有引用的 block
    // 返回被淘汰的 block 数量
    int EvictLRU(int n);

    // prefix lookup：给定请求的 block hash 链，返回能连续命中的最长前缀长度
    // 从第一个 block hash 开始，逐个检查 PrefixIndex 里有没有连续匹配
    int PrefixHitDepth(const vector<BlockHash>& req_block_hashes);

    // 动态调整容量：扩容时从 host memory 申请新 block，缩容时先驱逐再释放
    void ResizePool(int new_capacity);
};
```

关键设计：

- **BlockHash 对齐**：和 vLLM 一样，按前 ~256 token 一个 block 的粒度做递归哈希 `h_i = H(h_{i-1} ‖ token_i)`
- **in-flight 引用计数**：每个 block 有一个 `ref_count`，> 0 时 LRU 跳过；这是 vLLM 驱逐时不驱逐活跃请求 block 的核心机制
- **PrefixHitDepth 语义**：从第一个 block hash 开始连续匹配，断开后不计入（和 P1 的 prefixLen 语义一致，vLLM 也是这么做的）
- **线程安全**：用 `shared_mutex`——读路径（PrefixHitDepth）用读锁，写路径（Alloc/Ref/Unref/Evict）用写锁

实验与对读

实验（和 P2 的负载台联动）：

1. **LRU 驱逐曲线**：固定 pool 容量（比如 1024 blocks），用 P2 的负载台 trace 跑 1000 个请求，记录：驱逐命中率（被驱逐的 block 之后是否被重新引用）、in-flight block 占比随时间变化、prefix hit depth 下降曲线
2. **Prefix lookup 精度**：构造三种场景（全命中、半命中、全 miss），对比简化版和 vLLM 真实实现的 prefix hit depth 数值——简化版的 hash 算法和 vLLM 不一样，所以数值会有差异，但**曲线形状应该一致**（共享前缀比例越高，prefix hit depth 越高）
3. **Capacity 动态调整**：模拟弹性伸缩——pool 从 512 → 1024 → 512 → 2048，观察驱逐和 prefix hit depth 的响应

工业对读（必做）：精读 vLLM 内核里的驱逐逻辑——重点看它怎么处理"这个 block 还在 in-flight 请求里但又没被显式 ref"的边界情况。你的简化版用显式 Ref/Unref 处理，vLLM 内部是怎么处理的？差异有多大？

验收标准

- Block pool 全功能可用（Alloc/Ref/Unref/Evict/PrefixHitDepth/Resize）
- LRU 驱逐曲线 + prefix lookup 精度对比 + capacity 调整曲线三张图
- 《vLLM KV Cache 内核对读笔记》写完，能讲清楚简化版和 vLLM 内核的 3 个核心差异

回写 P2 README：一段「从消费路由层到理解引擎内核」的视角补齐说明，写清 M5、S7、M4 各补了

哪一块。

弹药三件套：
- 取舍：为什么 vLLM 用显式 ref_count 管理 in-flight block（而不是靠 GC 或所有权）；PrefixHitDepth 的语义为什么是连续匹配而不是覆盖率
- 坑：vLLM v1.x 的 block manager 接口和 v0.x 完全不一样（v0.x 用 `BlockTable` 二维数组，v1.x 用 `BlockPool + Index`），**版本锁定是第一天就要做的事**
- 数据：LRU 驱逐命中率曲线；**简化版 vs vLLM 内核差异对照表**

---

9. 微弹药 M1–M5（小品练手；M3/M4 已升级，M2 已删除）

> 2026-10-23 收口：M3 升级为独立仓库（1 周，作为 P1 附录），M4 升级为独立仓库（作为 P2 附录）；M1 只写笔记，M2 删除留档，M5 仍是笔记。编号不变，删掉的内容只打标记。

M1 mini-container（删减：只写一篇笔记，1 天，不开仓库）

2026-10-23 决定：不再写容器运行器。只写一篇笔记《hostPath 放弃了什么隔离》，里面必须有一个你亲手复现的隔离失效 case。笔记要回答：hostPath 把宿主机目录直接塞进 Pod，省掉了哪几层隔离；哪一类隔离失效会导致"一个 Pod 能看见或改到别的 Pod / 宿主机的数据"；这个 case 在什么条件下触发，怎么验证。

验收（2026-10-23 收口）：笔记里那个隔离失效 case 必须是可复现的——写清触发条件（哪条挂载、少了哪个 namespace）、验证命令和实际看到的现象，别人照着步骤能复现出来。

互补关系：P1 用 hostPath 把数据目录塞进 Pod，M1 补的是这一步到底放弃了哪些隔离。这比写一个 mini-container 更贴近 P1 的现勘和 S4 的存储叙事。

留档：原 mini-container 练习（2026-10-23 起不再执行）

> 下面是旧版内容，只留档，不排工期、不验收。

做什么：用 Go（或者直接用 C 调 `clone`）写一个 100–200 行的容器运行器。
- 用 `clone` 加 `CLONE_NEWPID/NEWUTS/NEWNS`（有余力再加 `NEWNET/NEWUSER`）；
- cgroup 用 v2：在 `cgroup.controllers` 体系下写 `cpu.max`、`memory.max`、`memory.swap.max`，把子进程的 pid 写入 `cgroup.procs`；
- rootfs 用 busybox 或 distroless 的根目录，调用 `pivot_root`（入门版可以先用 `chroot`）切换，再挂载 `/proc`、`/tmp`；
- stretch（可选）：用 overlayfs 的 lower/upper/merged 做一个可写层——这正好与 S2 里"overlayfs 上 O_DIRECT 的坑"呼应。

验收标准：容器内执行 `ps` 只能看到自己的进程树；hostname 相互独立；把 `memory.max` 设成 64M 后跑内存分配，能触发 cgroup OOM 并观察宿主行为；用 `lsns` 能看到新的命名空间。

你必须能讲：容器为什么不是虚拟机（共享宿主内核意味着什么、攻击面在哪里）；cgroup v2 相对 v1 的统一层级变化；pivot_root 与 chroot 的区别；为什么 user namespace 是 rootless 容器的关键；containerd/runC 在 K8s 链路中的位置（kubelet → CRI → containerd → runC）。

降级路径：`pivot_root` 卡住就退回 `chroot` 加 mount namespace，核心目标（找到 namespace/cgroup 的手感）不打折。

M2 mini-CNI（已删除，不执行，留档）

2026-10-23 决定：删除，不设复活条件。下面内容只留档，不排工期，v4.1 的 iptables/IPVS 扩展也一并停止。

互补关系：大项目只管容器调度（S6）和挂载（P1），不管 Pod 网络——原计划让 M2 补 bridge/veth/地址池/路由这套 Pod 网络的底层。

做什么：写一个能被 CNI 规范真正调用起来的最简网络插件（Go 可执行文件）。
- 实现 CNI spec 的 ADD / DEL / CHECK 三个命令：从 stdin 读 NetConf JSON，从 `CNI_ARGS` 和环境变量里拿 netns 路径；
- bridge 模式：建好 bridge，给容器 netns 创建 veth pair 并把一端塞进去，从 `/24` 地址池（用位图或租约文件持久化，防止 ADD 重启之后地址泄漏）分配 IP，配好接口地址与默认路由，再把 lo 拉起来；
- 用 `cnitool` 在两个独立的 netns 上验证互通；DEL 时释放 IP、删掉 veth。

验收标准：两个 netns 之间能 ping 通；重复 ADD 不会撞 IP；DEL 之后地址正确回收；用 `tcpdump` 能抓到 bridge 转发的包。

你必须能讲：K8s 的"IP-per-pod"网络模型；CNI 在容器创建链路中是哪个时刻被调用的（netns 已经建好、rootfs 已经就位之后，由运行时/CRI 调用）；跨节点流量怎么办（VXLAN、路由、BGP——这个练习故意不做，但你要能说清边界在哪里）；service 的 iptables/IPVS 与 CNI 为什么是两层东西。

降级路径：如果 CNI 调试吃掉了半天还没结果，就停下来，改为"读 containernetworking/plugins 的 bridge 插件源码，再用 `ip link/addr/route` 手敲复现全链路"，然后写笔记。

v4.1 扩展（2026-10-20 新增，+半天，随 M2 留档、不执行）：手写 iptables/IPVS 规则实现 mini-Service。动机：原范围里"Service 的 iptables/IPVS 是另一层"只是口头边界，面试官追问"你说那是另一层，你做过那层吗"时没货——这个扩展把这层也变成亲手做过的。在 M2 已跑通的 netns 环境上继续：
- 定一个 ClusterIP（如 `10.96.0.10:80`），手写 nat 表 DNAT 规则：`PREROUTING` 和 `OUTPUT` 两条链都要挂——本机发起的流量不走路由决策，这是第一个必踩的坑；
- 用 `-m statistic --mode random --probability 0.5` 挂两条 DNAT 分别指向两个后端 netns 的 IP——这正是 kube-proxy iptables 模式的微缩版（真集群里是 KUBE-SERVICES → KUBE-SVC-xxx → KUBE-SEP-xxx 三层链跳 + 按概率分摊）；
- 两个后端 netns 各起 `python3 -m http.server`，从宿主 curl ClusterIP 100 次，用 `iptables -t nat -L -v -n` 看每条规则的 pkts 计数——统计随机不是严格 1:1；
- 换 IPVS 模式再来一遍：`ipvsadm -A -t 10.96.0.10:80 -s rr` 加两个 real server（`-m` masq 模式），同样 curl 100 次，对比 rr 严格轮询与 iptables 统计随机的分布差；

- 量级实验：在 DNAT 规则前面塞 500 / 1000 / 5000 条 dummy 规则，用 wrk/ab 压两种模式的延迟——iptables 链式遍历 O(n) vs IPVS 哈希表 O(1)。单机量级很可能看不出差别，**阴性结果照样写进报告**。

扩展验收：ClusterIP 能轮询命中两个后端；iptables 计数器与 `ipvsadm -L --stats` 的分布对比有实测数字；dummy 规则实验有一张延迟表（哪怕是平的）。

扩展后必须能讲：kube-proxy iptables 模式的三层链跳结构与为什么用 statistic random 而非严格轮询；DNAT 发生在 netfilter 哪个钩子、conntrack 在这里记什么（回包如何找到路）；iptables 链式遍历 vs IPVS 哈希表在 Service 数量大时的差异；nftables 解决了 iptables 的哪些工程痛点（这次不做，能讲清边界即可）。

M3 mini-CSI（独立仓库，1 周，作为 P1 附录）

2026-10-23 决定：从微弹药升级成独立项目，给 1 周。必须真把 P1 从 hostPath 切到 CSI 上跑一轮，不是只做一个能挂载的 demo。

互补关系：P1 用 hostPath 只读挂载（preheat 模型），S4 是单机存储引擎——M3 补的是 CSI 标准 API，让你知道"把宿主目录挂到 Pod 上，K8s 标准流程是什么样的"，以及这套标准 API 到底要付多少开销。

做什么：写一个最小的 CSI 驱动，让它真的被 kubelet 调起来，再把 P1 的挂载方式从 hostPath 切过来跑一轮。
- 实现三个 gRPC service：Identity（GetPluginInfo/Capabilities/Probe）、Controller（最小实现，先不做 provisioning）、Node（NodeStageVolume 把宿主目录 bind mount 到全局挂载点；NodePublishVolume 再 bind mount 到 kubelet 给的 target path；NodeUnpublish/NodeUnstage 反向卸载）；
- 走 kubelet 的 plugin registration 机制：在 `/var/lib/kubelet/plugins_registry/` 暴露注册 socket，与 kubelet 完成握手（这一步本身就是最有价值的弹药）；
- 先打通 static PV 加你的 CSI driver 的绑定挂载闭环；stretch 目标：实现 CreateVolume/DeleteVolume（在宿主上建/删目录）加 StorageClass 动态供给；
- 把 P1 的挂载方式从 hostPath 切到你的 CSI，跑一轮和原来相同的 benchmark；
- 部署形态二选一，先跑通简单的那种：kind 里用 privileged DaemonSet，或者直接作为节点进程跑。

必须交出的数字：mount 延迟、kubelet 注册开销、重试语义差异、端到端 makespan 变化。

验收标准（2026-10-23 更新）：P1 从 hostPath 切到 CSI 后能跑通一轮 benchmark；上面四项数字都有；PVC 完成绑定，测试 Pod 挂载成功并能读写文件；删掉 Pod/PVC 之后 bind mount 被干净卸载（用 `findmnt` 验证没有残留）。

回写 P1 README："hostPath → CSI 的开销为 X ms/请求，makespan 变化 Y%。"

你必须能讲：CSI 三段式流程——Attach（ControllerPublish）、Stage（NodeStageVolume，块设备格式化并挂到全局挂载点）、Publish（bind mount 到 Pod）各自解决什么问题，你的实现为什么需要或不需要 Stage；kubelet registration 握手与 socket 的生命周期；ephemeral 卷与 persistent 卷的区别；Fluid/JuiceFS 为什么需要自己的 CSI driver（FUSE 挂载如何经由 CSI 注入 Pod）——最后这一问直接

和 P1 现勘连起来。

降级路径：动态供给做不完就停在 static PV 闭环（不影响主目标）；在 kind 里注册不上，就在节点上用 csc 工具直接调你的 gRPC 方法先验证，再回头解决注册。

M4 vLLM token 级 profiling（独立仓库，作为 P2 附录，1–2 天）

2026-10-23 决定：独立仓库，作为 P2 附录。顺序在 S7 之后，探针点由 M5 的精读笔记确定，不要自己拍脑袋选函数。

互补关系：P2 只有 vLLM /metrics 的**粗粒度**指标（TTFT/TBT/吞吐/前缀命中率）——M4 补的是 **token 级** 观测，能看到每个 token 的 prefill/decode 时间、prefix hit/miss 精确到 token 位置。

做什么：用 eBPF uprobes + Python 对 vLLM 做 token 级 profile。
- 用 `bpftrace` 或 `bcc` 的 uprobes 挂在 vLLM 的 Python 函数上（比如 `vllm/v1/core/async_scheduler.py` 里的 `schedule()` 方法、`gpu_model_runner.py` 里的 `execute_model()` 方法），或者用 `perf` 记录函数调用时间；
- 对每个请求记录：每个 prefill chunk 的耗时、每个 decode step 的耗时、哪个 replica 做的、prefix hit depth 精确到 token 位置；
- 用 Python 脚本解析采样数据，产出：**每个 token 的 prefill/decode 时间分布图**、**prefix hit/miss 的 token 位置热力图**、**preemption 发生在哪个 token 位置**。

> **不依赖 GPU**：如果当前是 CPU 后端的 vLLM，直接跑 uprobes；如果是 GPU 后端，用 uprobes 挂在 host Python 函数上（不需要挂 GPU 内核）。

和 P2 的联动：P2 的 TTFT/TBT 是"整个请求的聚合指标"，M4 是"每个 token 的拆解指标"——当 P2 的 TTFT 异常高时，用 M4 可以定位是"某个 prefill chunk 特别慢"还是"prefix miss 导致 prefill 重新计算"。

回写 P2 README：和 S7、M5 一起，补一段「从消费路由层到理解引擎内核」的说明。

面试时能讲：

> "P2 用 vLLM /metrics 看 TTFT/TBT 的粗粒度指标，我用 eBPF uprobes 做了 token 级 profiling——能看到每个 prefill chunk 和 decode step 的时间分布、prefix hit 精确到 token 位置。这个工具帮我定位了一次 TTFT 尖峰是因为 preemption 正好发生在 prefill chunk 边界，导致重新计算了 512 个 token 的 KV cache。"

M5 vLLM 引擎源码精读笔记（P2 引用，1–2 天）

2026-10-23 决定：只写笔记，不开仓库，作为 P2 的引用材料。它是 P2 线三步里的第一步，S7 和 M4 都要等它。

互补关系：P2 消费的是 AIBrix 路由层源码（Envoy plugin / Go router），没有读 vLLM 引擎内核——M5 补的是 vLLM 引擎核心的 C++/Python 源码精读，和 S7 联动（S7 是"读完之后写简化版"，M5 是"精读之后写笔记"）。

做什么：精读 vLLM 源码的三个核心文件（用 vLLM v1.x 锁定版本），产出一篇半页笔记：

1. **block_manager**：KV cache block 的分配/释放/淘汰/前缀索引——重点回答"vLLM 怎么判断这个 block 还能不能被 in-flight 请求引用"、"前缀命中深度是怎么从 block hash 链算出来的"
2. **scheduler**：continuous batching + chunked prefill + preemption 的调度逻辑——重点回答"vLLM v1 怎么平衡 prefill 和 decode 的 GPU 时间"、"preemption 怎么选被驱逐的请求"
3. **attention**：paged attention 的内存布局——重点回答"为什么 paged attention 要把 KV cache 分成固定大小的 block（比如 16 blocks/页）"、"token id 怎么映射到 block index"

和 P2 的联动：P2 的核心 2（弹性伸缩）需要理解"新副本需要预热多少 KV block 才能有意义地接受流量"——M5 的笔记里能找到 block 粒度和 prefix 命中深度的关系，直接给预热策略提供参数。

回写 P2 README：笔记里的关键结论（block 粒度、prefix 命中深度、M4 的探针点）要能直接被 P2 和 S7 引用。

面试时能讲：

> "P2 是网关视角看 KV cache 路由，我精读了 vLLM 引擎内核的 block_manager、scheduler、attention 三个核心模块——知道引擎内部怎么管理 KV block、怎么做 continuous batching、paged attention 的内存布局是什么样的。这让我能在路由层正确地消费引擎推的 KV 事件，也让我能理解为什么 PegaFlow 这种外部 KV cache 服务能工作。"

---

10. 微弹药与面试域的关系（v4 扩展）

微弹药	覆盖的面试域	联动
M1	hostPath 的隔离边界、Pod 挂载失效	P1 的挂载模型、S4 的存储叙事
M2（已删除）	原计划覆盖 CNI、容器网络、netns、iptables/IPVS；现在不执行	无
M3	CSI 标准 API 的代价测量、kubelet 卷生命周期、存储编排	P1 hostPath→CSI 切换、S4 存储岗叙事
M4	eBPF uprobes、token 级观测、profiling	P2 的 TTFT 归因、S7 的 token 级对比；排在 S7 之后
M5	vLLM 引擎内核、block_manager、scheduler、attention	S7（简化版 KV manager 的设计依据）、P2（引擎内核视角）；P2 线第一步

---

v4 定稿 *2026-10-15*，*v4.1* 砍 *S8 +* 删完成顺序/砍项清单 定稿 *2026-10-15*；*2026-10-23* 收口：按决策表重排，*S4/S3/M3* 聚焦缺口，*S6/S5* 并进 *P1 experimental/*，*S1* 定为论证题，*M1* 删减为笔记，*M2* 删

除留档。

内容由 AI 生成